Go Language Arrays

1️⃣ What is an Array in Go?

An **array** in Go is a **fixed-length sequence of elements of the same type** stored **contiguously in memory**.

var arr [5]int

Meaning:

- arr can hold **exactly 5 integers**
- Size is **part of the type**
- Memory is **allocated at compile time**

Index: 0 1 2 3 4

Value: 0 0 0 0 0

Zero values are automatically assigned.

2️⃣ Array Syntax Variations

Method 1 — Declaration

var arr [3]int

Output:

[0 0 0]

Method 2 — Declaration + Initialization

var arr = [3]int{10, 20, 30}

or

```
arr := [3]int{10, 20, 30}
```

Method 3 — Compiler Determines Size

```
arr := [...]int{10, 20, 30, 40}
```

Compiler counts elements.

Size = **4**

Method 4 — Index based initialization

```
arr := [5]int{
   0: 10,
   3: 40,
}
```

Result

[10 0 0 40 0]

3️⃣ Memory Layout of Arrays

Arrays are **contiguous blocks of memory**.

Example

```
arr := [3]int{10,20,30}
```

Memory:

Address	Value
0x1000	10
0x1008	20

0x1016 30

Because int = **8 bytes on 64-bit systems**.

Access formula:

Address = base + (index × size_of_type)

4️⃣ Array Length is Part of Type (VERY IMPORTANT)

In Go:

[3]int != [4]int

Example:

```
var a [3]int
var b [4]int


a = b // ❌ compile error
```

Error:

cannot use b (type [4]int) as type [3]int

Because they are **different types**.

5️⃣ Arrays are Value Types

Arrays are **copied when assigned**.

Example:

```
a := [3]int{1,2,3}
b := a
```

```
b[0] = 100

fmt.Println(a)
fmt.Println(b)
```

Output:

[1 2 3]

[100 2 3]

Memory:

a -> copy

b -> separate copy

Unlike slices.

6️⃣ Passing Arrays to Functions

Arrays are **copied when passed to functions**.

Example:

```
func modify(arr [3]int) {
   arr[0] = 100
}

func main() {
   a := [3]int{1,2,3}
   modify(a)
   fmt.Println(a)
```

```
}
```

Output:

[1 2 3]

Because **copy was modified**.

7️⃣ Passing Arrays by Pointer

To modify original array:

```
func modify(arr *[3]int) {
   arr[0] = 100
}


func main() {
   a := [3]int{1,2,3}
   modify(&a)
   fmt.Println(a)
}
```

Output:

[100 2 3]

8️⃣ Iterating Arrays

Method 1 — Classic loop

```
arr := [3]int{10,20,30}
```

```
for i := 0; i < len(arr); i++ {
   fmt.Println(arr[i])
}
```

Method 2 — range loop

```
for index, value := range arr {
   fmt.Println(index, value)
}
```

Ignore index

```
for _, value := range arr {
   fmt.Println(value)
}
```

9️⃣ Multidimensional Arrays

Go supports **arrays of arrays**.

Example:

```
var matrix [2][3]int
```

Structure

```
[
 [0 0 0]
 [0 0 0]
]
```

Initialization

```
matrix := [2][3]int{
  {1,2,3},
  {4,5,6},
}
```

Access

```
matrix[1][2] // 6
```

🔟 Comparing Arrays

Arrays can be compared **directly**.

```
a := [3]int{1,2,3}
b := [3]int{1,2,3}

fmt.Println(a == b)
```

Output

```
true
```

Rules:

- Same type
- Same length
- Same values

---

1 1️⃣ Arrays vs Slices (Important)

Feature	Array	Slice
Size	Fixed	Dynamic
Type	[3]int	[]int
Copy	Value copy	Reference
Memory	Stack usually	Heap
Usage	Rare	Very common

Example slice:

```
s := []int{1,2,3}
```

---

1 2 Arrays and Pointers

```
arr := [3]int{10,20,30}


p := &arr
```

Access:

```
(*p)[1]
```

Go also allows:

```
p[1]
```

Go **auto dereferences**.

---

1 3 Arrays Inside Structs

```
type Data struct {
   values [3]int
```

```
}

d := Data{
   values: [3]int{1,2,3},
}
```

---

1 4 Array of Structs

```
type User struct {
   name string
}

users := [2]User{
   {"Alice"},
   {"Bob"},
}
```

---

1 5 Real Go Runtime Behavior

Arrays are mostly used:

- internally
- fixed size buffers
- cryptography
- performance critical code

Example:

[32]byte -> SHA256

[16]byte -> UUID

Example:

var hash [32]byte

---

1 6 When Should You Use Arrays?

Use arrays when:

✓ fixed size known at compile time
✓ memory layout matters
✓ performance critical
✓ low-level systems programming

Example:

network packets

hash storage

binary parsing

---

1 7 Why Arrays Are Rarely Used

Most Go programs use **Slices instead**.

Because slices:

- grow dynamically
- share memory
- easier APIs

Example:

numbers := []int{1,2,3}

numbers = append(numbers, 4)

Arrays **cannot grow**.

---

1 💡 Advanced Example (Deep Understanding)

```
package main

import "fmt"

func main() {

    arr := [3]int{10, 20, 30}

    modify(arr)
    fmt.Println("after modify:", arr)

    modifyPointer(&arr)
    fmt.Println("after pointer modify:", arr)
}

func modify(a [3]int) {
    a[0] = 999
}
```

```
func modifyPointer(a *[3]int) {
   a[0] = 888
}
```

Output

```
after modify: [10 20 30]
after pointer modify: [888 20 30]
```

---

2 0 Visual Diagram

Array

```
[10 20 30]
```

Memory

```
+----+----+----+
|10  |20  |30  |
+----+----+----+
```

Index

```
 0   1   2
```